

Information retrieval is the art and science of searching for information in documents, searching for documents themselves, searching for metadata which describe documents, or searching within relational databases or hypertext networked databases such as the Internet for text, sound, images or data, [Wikipedia, 2005].

The newly defined term of *situation retrieval* describes specialized information retrieval within repositories and reports of SITUATIONALSTATEMENTS. In chapter 4 we have demonstrated that the model of the statements is independent from the actual used syntactical realization and in chapter 5 we have demonstrated that the model is even decoupled from the semantics which is moved to the ontology. For the information retrieval part this insight leads to the threefold relationship: one with the model, one with the syntax and one with the semantics. Since *context logs*, *user journals* and especially *user models* are also represented in this statement framework, situation retrieval directly covers the task of querying these logs, journals and models.

In this chapter, we first present the model of SITUATIONALQUERIES with its three groups of attributes. After that, we present the model of SITUATIONREQUESTS, followed by the query algorithm. Then, two sections about UserQL and conflict resolution follow. The architecture of situation retrieval with conflict resolution will be described in subsection 6.3.6, while the discussion on the design decisions for UserQL can be found in section 6.4. The integration into the overall architecture of the USERMODELSERVICE and into the procedural view to the USERMODELSERVICE will be explained in chapter 8 in the figures 8.1 and 8.2.

6.1 The Model of SITUATIONALQUERIES

SITUATIONALQUERIES form the counterpart to SITUATIONALSTATEMENTS, since each situation attribute finds a corresponding attribute in the model of SITUATIONALQUERIES. SITUATIONALQUERIES consist of three identified groups of attributes: the *match*-attributes, the *filter*-attributes and the *control*-attributes. They are named after three macro-steps in the query evaluation process as displayed in figure 6.2. The *select*-attribute is modeled together with the *control*-attributes. The three groups of attributes are described as boxes below.

6.1.1 The Match Box

Match
Subject
Auxiliary
Predicate
Range
Object
...

The match-attributes are named `subject`, `auxiliary`, `predicate`, `range`, `object` and so on, exactly after their corresponding attributes within the `SITUATIONALSTATEMENTS`. Actually any attribute from the model of statements can be used as match attribute in the model of queries. To differentiate between the two sets, the dot-notation is introduced as in `statement.subject` versus `query.subject`. When the *matching algorithm* is applied, only statements that match all given match-attributes are handed over to the *filtering algorithm*. Furthermore

figure 6.4 - which shows the matching algorithm - reveals that it can be more complex than just the comparison for equality: semantic functionality like the *extension relation* can be applied. This relation maps to each class its subsumed instances. For example if the match attribute `query.subject = extension(person)` is given, all statements with `statement.subject ∈ {Boris, Jörg, Margeritta, ...}` are matched, namely instances that belong to the class (or the subclasses) of *person*. This elegant mechanism to integrate semantic inference into the syntax of the query language has already been prepared by the extended RDF resource concept of *UbiExpression*, as described in section 4.5 from page 74 onwards.

6.1.2 The Filter Box

Filter
Requestor
Intention
minConfidence
maxConfidence
fromTime
untilTime

Filter attributes set further restrictions on the returned statements. There are currently three filters implemented: a *privacy filter*, a *confidence filter* and a *temporal filter*. Figure 6.5 defines all three filters within the complete *filtering algorithm*. The attributes `query.requestor` and `query.intention` decide according to the attributes `statement.access`, `statement.purpose` and `statement.owner` if the privacy filter can be passed. If no permission is granted, the statement will be filtered out, even though it might

contain the answer to the query. The privacy filter needs no activation since it is always activated. If no `requestor` and `purpose` attributes are stated in the query, the values `requestor=anonymous` and `purpose=commercial` are set as defaults, which results in a minimal access in privacy terms. The confidence attributes `minConfidence` and `maxConfidence` allow us to restrict the confidence value to a certain interval. For example it might be interesting to analyze only statements that carry a higher confidence value than 0.75 within the range [0, 1]. The confidence filter will be used automatically, if at least one of the attribute-value pairs `minConfidence=value` or `maxConfidence=value` is used. The *temporal filter* allows to put constraints on the statements' temporal aspects, since in our approach no statement is deleted, but either marked as *replaced* or marked as *expired* if it should not be used any more. This especially allows to neglect older statements with the attribute `fromTime`, or the other way round, to allow for historic views by filtering out all statements after a certain point of time with the attribute `untilTime`. The temporal filter can be activated by adding `fromTime=time` or `untilTime=time` to the query. To summarize, the filter box attributes lead to a fine grained mechanism for extending the expressivity of the situation retrieval process.

6.1.3 The Control Box



The attribute repository controls the select-step by defining the repository (or the set of repositories) to which the query is being applied to. The addressing schema to the decentralized and distributed UserML-repositories is defined in section 6.2.2. During the control-step, possible conflicts are resolved and the appearance of the returned UserML document is defined. The conflict resolution can be guided by the attributes strategy and ranking and the appearance of the result is influenced by the two attributes format and naming. The attribute function allows to post-process the so far calculated set of statements into any kind of result like an average number for example. The control-step procedures and especially the conflict resolution strategies are analyzed in section 6.3.

6.1.4 The Model of SITUATIONREQUESTS

A list of SITUATIONALQUERIES is called a SITUATIONREQUEST, see figure 6.1 for a schematic view. A SITUATIONREQUEST is sent to the user model and context service that resolves each query after the other in a row (or if possible in parallel) and returns the resulting SITUATIONREPORT.

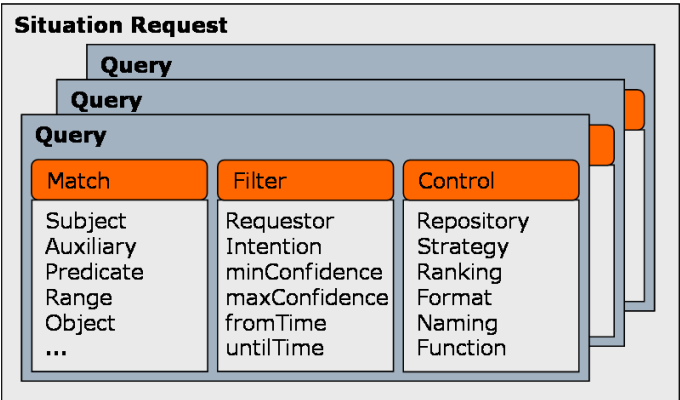


Figure 6.1: A SITUATIONREQUEST consists of a set of SITUATIONALQUERIES

- The naming convention of the two introduced concepts is defined in the following listing:
- A SITUATIONALQUERY is a collection of attributes as defined in table 6.1. The term QUERY without the prefix is used as shortcut.
 - A SITUATIONREQUEST is a collection of SITUATIONALQUERIES, that belong to the same situational description. The term REQUEST without the prefix is used as shortcut.
- Table 6.1 shows a detailed description of the intended attribute meanings within a SITUATIONALQUERY.

Match	<i>all attributes introduced for SituationalStatements</i>
subject	selecting the main statement entity, <i>default: any</i>
auxiliary	selecting the auxiliary part of the property, <i>default: any</i>
predicate	selecting the predicate part of the property, <i>default: any</i>
range	selecting the range part of the property, <i>default: any</i>
object	selecting the object, <i>default: any</i>
id	selecting the statement by id, <i>default: any</i>
group	selecting the group of statements, <i>default: UserModel</i>
location	selecting the spatial extension of the statements, <i>default: any</i>
⋮	see table 4.1 for a complete list of attributes since every situational statement attribute can be used as a matching attribute in the query

Filter	<i>a collection of filter attributes</i>
requestor	the requesting user or system, <i>default: anonymous</i>
intention	what is intended to be done with the statement, <i>default: commercial</i>
minConfidence	minimal confidence value that must hold, <i>default: 0</i>
maxConfidence	maximal confidence value that must hold, <i>default: 1</i>
fromTime	start of the time interval, <i>default: whenever</i>
untilTime	end of the time interval, <i>default: now</i>

Control	<i>a collection of control attributes</i>
repository	the chosen, respondent situation container, <i>default: system's choice</i>
strategy	conflict resolution strategies, <i>default: latestOnly</i>
ranking	sorting and ranking of the results, <i>default: newestFirst</i>
naming	manipulating the appearance of the names, <i>default: longName</i>
format	manipulating the appearance of the XML format, <i>default: UserML</i>
function	applying evaluation functions to the results, <i>default: none</i>

Table 6.1: Attributes of SituationalQueries with default values

6.1.5 The Query-Answer Algorithm

Figure 6.2 shows the concept of the macro-steps in the query evaluation process. The *select*-step chooses the report or repository to which the query is applied. The *match*-step returns all statements that match the corresponding query attributes. The *filter*-step filters out further unwanted statements, while the *control*-step performs conflict-resolution and transforms the final statements into the returned result. A query model as defined in section 6.1 needs to carry all the intended attributes and parameters to allow for this multi-step query evaluation process.

Figure 6.3 defines the query-answer algorithm as a sequence of the *selecting procedure*, *matching procedure*, *filtering procedure* and *control procedure*: after the selection of the repository (as shown in figure 6.2) has been done by the user model and context service in

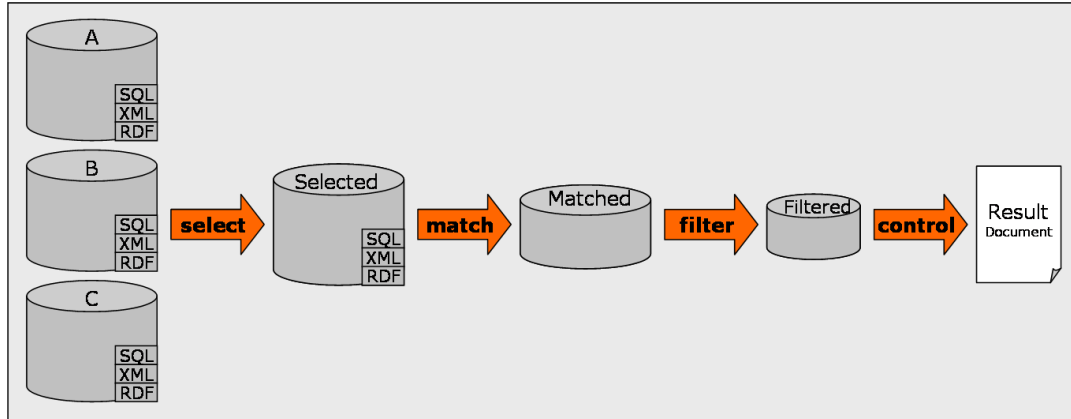


Figure 6.2: Situation retrieval with the macro steps: Select, Match, Filter and Control

the *selecting procedure*, the *answering procedure* linearizes the three remaining macro steps and returns the reduced set of statements in each step.

```

procedure answering (query, USERMODELSERVICE)
  SELECTINGRESULT := selecting (query, USERMODELSERVICE)
  MATCHINGRESULT := matching (query, SELECTINGRESULT)
  FILTERINGRESULT := filtering (query, MATCHINGRESULT)
  return RESULT := control (query, FILTERINGRESULT)
  
```

Figure 6.3: The linearized answering procedure

The *matching procedure* as shown in figure 6.4 compares all given match attributes with the corresponding statement attributes. Furthermore it integrates semantic functionality, here shown by the two examples: ontological *extension* and spatial *inclusion*. Further semantic functionality that could have interesting applications are for example the ontological *sameAs* and the spatial *closeBy*.

The *filtering procedure* as shown in figure 6.5 operates on the MATCHINGRESULT. Each statement is individually checked if it passes the *privacy filter*, the *confidence filter* and the *temporal filter*. The *privacy filter* checks if the `statement.access` is either set to **public**, or if it is set to **friends**, that the *friends relation* holds between the `query.requestor` and the `statement.owner`, or if it is set to **private** that the `query.requestor` is the same as `statement.owner`. Further filters can be added to this part of the algorithm. Especially a fine-grained spatial filter that covers more than the *spatial inclusion relation* (which is already realized by the match attributes) might become important.

The next section introduces two syntactical realizations of this SITUATIONALQUERY model.

```

procedure matching (query, SELECTEDREPOSITORY)
  forall statement  $\in$  REPOSITORY
    begin
      if statement.subject = query.subject
        or statement.subject  $\in$  extension( query.subject )
       $\wedge$  statement.auxiliary = query.auxiliary
       $\wedge$  (statement.predicate = query.predicate
        or statement.predicate  $\in$  extension( query.predicate ))
       $\wedge$  statement.range = query.range
       $\wedge$  statement.object = query.object
       $\wedge$  statement.id = query.id
       $\wedge$  statement.group = query.group
       $\wedge$  statement.location = query.location
        or statement.location  $\in$  inclusion( query.location )
       $\wedge$  ...
      then add statement to MATCHINGRESULT
    end
  return MATCHINGRESULT

```

Figure 6.4: The matching procedure with integrated semantic functionality

6.2 The Syntax of SituationQL and UserQL

SituationQL defines instances of a *Situation Query Language* and the acronym UserQL stands for *User Model Query Language*. Both are syntactically equal and form the main concept in situation retrieval with SITUATIONALSTATEMENTS. They form the counterparts to SituationML and UserML. Since the USERMODELSERVICE which is introduced in section 8.1 is implemented as an HTTP web server, there exist the two methods *post* and *get* to send query information to the server. This results in two versions of the query language, namely UserQL/XML and UserQL/URI. The first one defines the queries in XML format and can be sent via the post method to the server. The second one defines the queries in URI format and can be sent via the get method to the server.

6.2.1 The UserQL/XML Query Language

The listing in figure 6.6 presents the default SituationQL/XML representation for SITUATIONALQUERIES. The root element is named <query>. All attributes are defined by XML-elements with their original name, while the corresponding attribute groups like *match* or *control* are omitted. The flat tree structure is the easiest for prototyping and can directly be transformed into a relational data model for database storage. SITUATIONAL-QUERIES define up to 37 attributes, however the average query will be short, since empty elements can be omitted. The variables from q1 to q37 can carry ordinary RDF node values but also more complex *UbiExpressions* as defined in section 4.5.

```

procedure filtering (query, MATCHINGRESULT)
  forall statement  $\in$  MATCHINGRESULT
    begin
      if ( statement.access = public
        or ( statement.access = friends
           $\wedge$  query.requestor  $\in$  friends (statement.owner) )
        or ( statement.access = private
           $\wedge$  query.requestor = statement.owner ))
      and statement.confidence  $\geq$  query.minConfidence
         $\wedge$  statement.confidence  $\leq$  query.maxConfidence
      and statement.start  $\geq$  query.fromTime
         $\wedge$  statement.start  $\leq$  query.untilTime
      then add statement to FILTERINGRESULT
    end
  return FILTERINGRESULT

```

Figure 6.5: The filtering algorithm, using the externally defined friends-relation

The XML application UserQL/XML forms only one concrete XML-instance of the model of SITUATIONALQUERIES. Syntactic variations of UserQL/XML could be defined if they promise advantages. Figure A.2 on page 215 shows for example the XML-Schema tree for an alternative realization.

UserQL/XML means that the query is represented in XML, but not necessarily the repositories. They are mostly stored in relational databases. However, XML repositories could be treated with XSL and XPath, a language for addressing parts of an XML document. The XPath data model provides a tree representation of XML documents. The result of an XPath expression may be a selection of nodes from the input documents, or an atomic value, or more generally, any sequence allowed by the data model, see W3C for a documentation.

6.2.2 The UserQL/URI Query Language

The UserQL/URI query language defines a structured subset of all possible URIs. Thus the attribute names and especially the attribute values have to fulfill the constraints that are implied by using URIs, see [Berners-Lee et al., 1998]. Figure 6.7 shows the partial definition of UserQL/URI in BNF-format, where the terminals are surrounded by ' -signs. In general, URIs allow us to define a list of attribute value pairs, the so-called *query string*, that is divided from the URL base with the ?-sign. The pairs are separated by &-signs, while the attribute name and the attribute value, a so-called *segment*, is syntactically separated by the =-sign. New in our extended query string approach for UserQL/URI is the additional evaluation of the | -sign as second-level list element within one single attribute value. The syntax is inherited from the EBNF grammar of UbiExpressions where for example the subject can be a | -sign-separated list of entities as in `subject=Boris|Margeritta|Christian`. The semantics for this list can be interpreted as logical **or**. The nested list structure and its semantics is defined in a relational model in section 7.1.1 on page 142.

```

<query>
  <!-- Match Attributes -->
  <subject>      q1    </subject>
  <auxiliary>    q2    </auxiliary>
  <predicate>    q3    </predicate>
  <range>        q4    </range>
  ...
  <unique>       q22   </unique>
  <replaces>     q23   </replaces>
  <group>        q24   </group>
  <notes>        q25   </notes>

  <!-- Filter Attributes -->
  <requestor>    q26   </requestor>
  <intention>    q27   </intention>
  <minConfidence> q28   </minConfidence>
  <maxConfidence> q29   </maxConfidence>
  <fromTime>     q30   </fromTime>
  <untilTime>    q31   </untilTime>

  <!-- Control Attributes -->
  <repository>   q32   </repository>
  <strategy>     q33   </strategy>
  <ranking>      q34   </ranking>
  <naming>       q35   </naming>
  <format>       q36   </format>
  <function>     q37   </function>
</query>

```

Figure 6.6: *SituationQL/XML is the default representation SITUATIONALQUERIES*

6.2.3 Example Queries with UserQL/URI

This subsection presents three example queries to the `u2m.org` USERMODELSERVICE in UserQL/URI format.

A) If one wants to ask *What is the age of Boris?* to the USERMODELSERVICE, one first has to identify the subject and the property of this question. In this example, the subject could be identified as “Boris” and the property could be described as “has age”. Due to the design decisions in SITUATIONALSTATEMENTS, the property “has age” can be defined by three variables: the auxiliary *hasProperty*, the predicate *age* and a range like *integer* or *age group*. The default value for the range can also be defined in the general user model ontology GUMO and thus be omitted in the query. Figure 6.8 shows the identified attribute-value pairs. Figure 6.9 shows the query in UserQL/URI format.

String names like *Boris* or *age* are not case sensitive, it does not matter if they are written in upper case letters or lower case letters. However, since using the string names for referring to the intended elements is in general not sufficient to refer uniquely, the use of *UbisIds* as shown in figure 6.10 is also possible.

An interesting aspect about this user model request is that it could probably be answered without having a direct entry about *Boris’ age* in the requested user model repository. The


```

UserQL/URI    --> 'http://www.u2m.org/service.php'
               '? subject      =' Q1
               '& auxiliary    =' Q2
               '& predicate    =' Q3
               ...
               '& requestor    =' Q26
               '& intention    =' Q27
               ...
               '& repository    =' Q32
               '& strategy      =' Q33
               ...
               '& format        =' Q36
               '& function      =' Q37
Q1, ..., Q37  --> UbiExpression

```

Figure 6.7: Partial EBNF grammar definition for *UserQL/URI*. The succeeding EBNF grammar for *UbiExpression* can be found in section 4.5.3 on page 77

```

query.subject = Boris
query.auxiliary = hasProperty
query.predicate = age

```

Figure 6.8: The attribute-value pairs that represent the query “What is the age of Boris?”

USERMODELSERVICE could deduce this information for example from other information like *his birthday* or from the *ages of his elder and younger sisters*. Even though these two described inferences appear fairly simple they point out that *UserQL* queries applied to the USERMODELSERVICE differs from ordinary SQL queries applied to general databases. The interrelationship between the syntax and ontological reasoning in our approach will be discussed in chapter 7. On the other hand, instead of having no entry about *Boris’ age* in the repository, there could be several conflicting entries by different *creators* with probably different *confidence* values. In such a case the USERMODELSERVICE has to resolve the answer by a given conflict resolution strategy. The challenge of conflict resolution is discussed in the following section 6.3.

B) If one is interested in a query like *What do you know about the personality of Margeritta?*, the predicate focuses on a whole class of user model dimensions, namely the *personality* entries like *extraversion*, *introversion*, *judging* or *perceiving*. The query syntax in *UserQL/URI* is the same as in example A), only that the resource identifier of the *predicate* attribute now refers to a class of concepts instead of to an instance. Figure 6.11 shows the corresponding URI request.

These requests are related to the structured query language SQL queries. Since the whole SITUATIONALSTATEMENTS are returned, it could be compared to a *SELECT * FROM (http://www.u2m.org/service.php) WHERE subject=Margeritta AND auxiliary=hasProerty AND predicate=Personality*. However, apart from the syntactical difference, there is a difference is the evaluation of the last attribute-value pair, since *Personality* is detected as

```
http://www.u2m.org/service.php?
subject=Boris&auxiliary=hasProperty&predicate=Age
```

Figure 6.9: The *UserQL/URI* that represents the query “What is the age of Boris?”

```
http://www.u2m.org/service.php?
subject=210002&auxiliary=600100&predicate=800302
```

Figure 6.10: The *UserQL/URI* that also represents the query “What is the age of Boris?”

a class and not as an individual, the ontological *extension* function is executed and the output result would represent the following triples, (compare with figure 8.13):

```
subject=Margeritta, predicate=open-minded, object=high
subject=Margeritta, predicate=indulgent, object=low
subject=Margeritta, predicate=tempered, object=high
subject=Margeritta, predicate=optimistic, object=medium
```

C) In the request *Tell me all interests of Christian!* one can omit the `predicate` attribute because the `auxiliary` attribute is sufficient, as shown in figure 6.12. Omitting an attribute in general means that there are no restrictions set on this attribute. Thus, if one omits all attributes, all entries of the `USERMODELSERVICE` will be returned according to the default strategy, format and naming settings.

6.2.4 How to Manipulate the Output Format?

This subsection presents some settings for manipulating the output format of the returned document. The two query attributes `query.format` and `query.naming` are discussed. The overall structure of the XML document can be chosen with the attribute `format`, while the attribute `naming` changes the appearance of the variables. “Boris’ Age Example” with the chosen naming convention `nameWithParent` and the selected format `UserMLrdf` is shown in figure 6.13 below.

Table 6.2 shows the predefined values for the `UserQL` attribute `naming`. Compare the section about extended resource identification 4.5 on page 74 and the XML-Schema of `SITUATIONREQUESTS` in appendix A.3.3 on page 215.

The default setting for the `naming`-attribute is `longName`, which combines the written name and the id with a dot. Further examples for such “long names” are `BloodPressure.800131` and `Gender.800300`. Table 6.2 shows the predefined values for the `UserQL` attribute `format`. Compare the section about the syntax of `SituationML` and `UserML` in section 4.4 on page 67.

```
http://www.u2m.org/service.php?
subject=Margeritta&auxiliary=hasProperty&predicate=Personality
```

Figure 6.11: This URI represents the request “Tell me all about the personality of Margeritta”

```
http://www.u2m.org/service.php?
subject=Christian&auxiliary=hasInterest
```

Figure 6.12: This UserQL/URI represents the query “Tell me all interests of Christian”

```
http://www.u2m.org/service.php?
subject=Boris&auxiliary=hasProperty&predicate=Age
&naming=nameWithParent&format=UserMLrdf
```

Figure 6.13: “What is the age of Boris?” with an alternative *naming* and *format* attribute

Attribute	Value	Description
naming	id	unique numerical identifier in the format of a positive integer between 100000 and 999999
naming	label	textual identifier, possibly with blanks
naming	shortName	transformed label without blanks and only uppercase letters at the beginning of each new word
naming	longName	(default) combines the shortName with the id in the following way: “shortName.id”
naming	nameWithParent	combines the shortName with the shortName of the parent element as prefix: “shortName(parent).shortName”
naming	resource	unique RDF resource, combining a URI with the fragment identifier longName: “URI#longName” or an RDF literal

Table 6.2: Possible values for the UserQL attribute *naming*

Attribute	Value	Description
format	UserML	(default) UserML definition where the <statement> element carries all information in its up to 25 subelements
format	UserMLmin	minimal variation of UserML where the <statement> element carries all information in its attributes
format	UserMLmax	XML application with the main element <statement> and its five subelements <mainpart>, <situation>, <explanation>, <privacy> and <administration>.
format	UserMLrdf	RDF representation of the same semantic information as in UserML
format	PredicateObject	returns only predicate-object pairs as a newlined list

Table 6.3: Possible values for the query-attribute *format*

6.3 Conflict Resolution

In widely spread distributed user modeling and context awareness one can not expect that all systems use the same standards for representing user models and context information. However, as a common basis and to narrow down the research area, we assume that all systems, that are involved in the information exchange, are able to use the framework of SITUATIONALSTATEMENTS: either as their main representation language or at least as an additional transforming feature. Even though we assume that all involved user-adaptive and context aware systems are supposed to communicate with the same *grammatical* framework of SituationML and SituationQL, there is still a whole list of interesting points of possible misunderstandings left to be solved. Systems that do not support SituationML and SituationQL can still cooperate with such systems on the level of SITUATIONALSTATEMENTS in such a way that they consume or contribute new statements, or they could be connected with the help of specialized XSL transformations that have to be designed individually.

6.3.1 Situational Conflict Categories

As every user and every system is allowed to enter statements into repositories, some of this information might be contradictory. Conflicts among SITUATIONALSTATEMENTS like for example a contradiction caused by different opinions of different creators or changed values over time are categorized in the following listing.

1. ON THE REPRESENTATIONAL LEVEL:
each system can choose between a variety of possible representations to express the same information which leads to the so-called *variation mapping*
2. ON THE SYNTACTICAL LEVEL:
statements can for instance differ in the use of the statement attributes like `subject`, `predicate`, `object`, `range`, `start` etc., clear *modeling guidelines* are necessary.
3. ON THE SEMANTICAL LEVEL:
the systems are not forced to use the same vocabulary, that is to say the same ontology, to represent the meaning of the concepts, which leads to the user model integration problem number one: *ontology merging* and *semantic web integration*.
4. ON THE OBSERVATION AND INFERENCE LEVEL:
several sensors can see same things differently and claim to be right, measurement errors can occur, systems may have preferred information sources
5. ON THE TEMPORAL AND SPATIAL LEVEL:
information can be out of date or out of spatial range, a degree of expiry can hold, thus reasoning on temporal and spatial meta data becomes necessary
6. ON THE PRIVACY AND TRUST LEVEL:
information can be hidden, incomplete, secret or falsified on purpose, a system of trustworthiness could be applied

All these aspects are content- and domain independent and can be relevant for user related as well as context related information. In the remaining part of this chapter, several conflicts are analyzed and categorized and possible solutions are presented. In relation to the situation retrieval task, the two concepts of *precision* (*How many retrieved statements are really relevant?*) and *recall* (*Is all relevant information retrieved?*) are also taken into consideration. This section is partly based on the research described in [Blass, 2004] and on ideas about meta rules and conflict resolution in OPS5, see for example [Brownston et al., 1985]. *Conflict resolution* forms the first part of the *control* macro-step as shown in figure 6.2.

6.3.2 Prefacing Examples with Conflicting Statements

A) Imagine you should play chess against user A and you have the following rule of thumb:

*if the opponent is a good player, you start with a defensive opening
but if the opponent is a weak player, you choose an offensive opening.*

You have heard the three following, conflicting statements about user A's ability to play chess: System C claims that A is a good player, system B claims that A is a medium good player while user A claims that she/he himself is a bad player. Figure 6.14 visualizes these

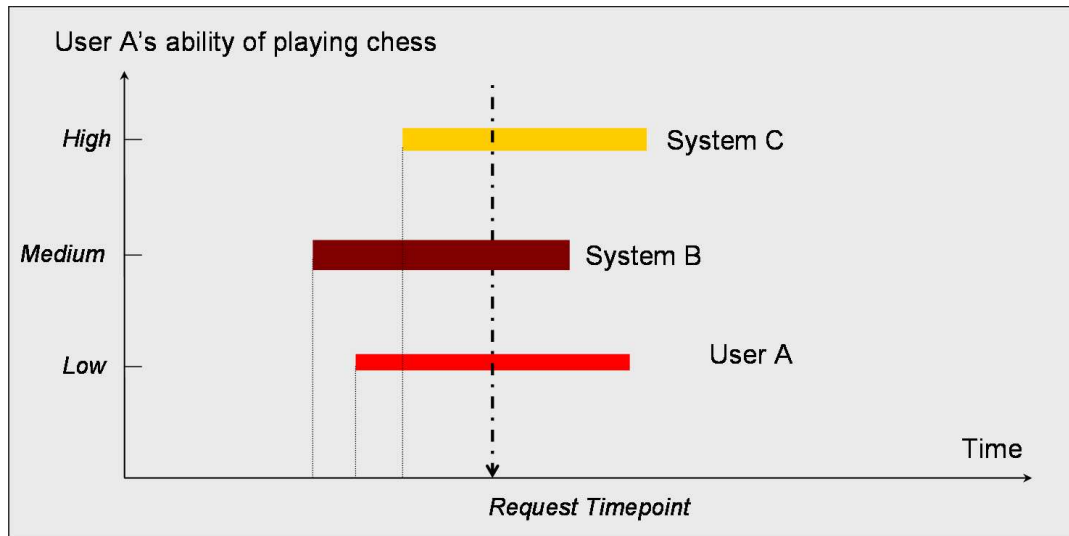


Figure 6.14: Example visualization of three conflicting statements

three statements as rectangles that also indicate additional meta data dimensions: the time-axis shows the `start`, `end`, `durability` and `expiry` values which reveals that system B's statement is the oldest one, while system C's statement is the most recent one. The height of the rectangle relates to the `confidence` value: the confidence of system B's statement is the highest while the user's statement has the lowest confidence. So, how should we decide?

How do we resolve such conflicts? Naive approaches could ask: should we return the latest entry? Should we return a random element? Should we return the one with the highest

confidence value or should we prefer the user's entry? Conflicting statements turned out to be a complex and general problem but with no general solution that is valid for all situations. The measures of belief that are based on evidences in the *Dempster-Shafer theory*, see section 4.2, could lead to a sophisticated solution. However, in our approach we support the means to express the intended conflict resolution methods as described in the next subsection. Conflicts on the semantic level are presented in the the following example.

B) Let us assume that a resource-adaptive mobile device uses the following energy saving and contrast optimizing adaptation rule if its device's battery is low:

*if the surrounding brightness is low then set the display brightness to its minimum,
if the surrounding brightness is medium or high then set the display brightness to medium.*

Since the mobile device has no light sensor by itself, it needs to retrieve and infer information about the *surrounding brightness* from the user model and context service. The current position of the mobile device should be *Room124* and the light situation there is given by the semantically conflicting statements:

- 1) subject=*Room124* auxiliary=*hasProperty* predicate=*brightness* range=*lowHigh* object=*low*
- 2) subject=*Saarbrücken* auxiliary=*hasProperty* predicate=*weather* object=*sunny with no clouds*
- 3) subject=*Light124.1* auxiliary=*hasProperty* predicate=*switched* range=*onOff* object=*on*

Figure 6.15: Example statements that form a semantic conflict

The most direct statement 1) about the brightness in *Room124* claims that it is dark. However, statement 2) claims that the sun is shining. Since the device is inside a building which could hinder the sun to lighten the surroundings the situation is not clear. Furthermore statement 3) claims that the lights in this room are switched on. This conflict is difficult to detect since ontological reasoning, spatial reasoning and qualitative reasoning might be necessary to handle this problem satisfactorily. Subsection 6.3.5 introduces a mechanism to detect such semantical conflicts.

6.3.3 Conflict Resolvers and Conflict Resolution Strategies

Conflict Resolvers are a special kind of filter that control the *conflict resolution process*. An ordered list of these resolvers defines the *conflict resolution strategy*. They are modeled in the `query.strategy` attribute. These resolvers are needed if the *match process* and *filter process* leave several conflicting statements as possible answers. Three kinds of conflict resolvers can be identified: the *most(n)*-resolvers that use meta data for their decision, the *add*-resolvers that add expired or replaced statements to the conflict sets, and the *return*-resolvers that don't use any data for their selection.

mostRecent(n) Especially where sensors send new statements on a frequent basis, values tend to change quicker than they expire. This leads to conflicting non-expired statements. The *mostRecent(n)* resolver returns the *n* newest non-expired statements, where *n* is a natural number between 1 and the number of remaining statements.

- mostNamed(n)** If there are many statements that claim A and only a few claim B or something else, than n of the “most named” statements are returned. Of course it is not certain that the majority necessarily tells the truth but it could be a reasonable rule of thumb for some cases.
- mostConfident(n)** If the confidence values of several conflicting statements can be compared with each other, it seems to be an obvious decision to return the n statements with the highest confidence value.
- mostSpecific(n)** If the range or the object of a statement is more specific than in others, the n “most specific” statements are returned by this resolver. For example if: `auxiliary=hasKnowledge`, `predicate=chess` and first `range=yesNo` while the second `range=Novice-Occasional-Professional-Expert-Grandmaster`, the statement with the second range contains a more specific information. Another specificity range ordering is for example: `yesNo < lowMediumHigh < 0%-100%`
- mostPersonal(n)** If the `creator` of the statement is the same as the statement’s subject (a self-reflecting statement), this statement is preferred by the *mostPersonal(n)* resolver. Furthermore, if an *is-friend-of relation* is defined, statements by friends could be preferred to statements by others. However, this resolver bears the problem, that users might not be their best judge. However due to privacy arguments, the user’s own statements that are given (on purpose) should be preferred. (An alternative approach with the `creator` information could have been to define a *trusted-creator relation*.)
- addExpired** Per default the already expired statements are filtered out. However, if one wants to take them into consideration, the *addExpired*-resolver adds these statements to the conflict sets.
- addReplaced** Statements that are marked with the replaced-flag by other statements, are also per default filtered out and not considered in the situation retrieval process. The *addReplaced*-resolver brings these statements back into the process.
- addPrivate** Statements that do not pass the privacy settings are always filtered out. However, for development, testing and administrative reasons experimental private statements may also be recognized with the *addPrivate*-resolver.
- returnAll** If the remaining conflict set should not be resolved any further by the integrated mechanism, the resolver *returnAll* returns all remaining statements that can then be resolved by an external conflict resolution method, resolved by introspection or left unresolved since our approach also allows conflicting extensions in parallel.
- returnNone** If there still occurs a conflict that could not be resolved until the *returnNone* resolver is applied, no statement is returned. This is a very safe way not to say something wrong. This rule could be compared with *sceptical inheritance* in non-monotonic reasoning: *I don’t know!*
- returnRandom(n)** if after applying several filters still no unique value is found but a unique answer is expected, a random pick will be offered by this resolver. This credulous behavior is selected by the requestor and therefore acceptable.

returnDialog if no unique value is found, an alternative conflict resolution strategy could be *clarification by dialog*¹. In some cases an appropriate human-computer dialog will be initiated in this case.

These conflict resolver rules are based on common sense heuristics. An important issue to keep in mind is the problem that resolvers and strategies imply uncertainty. To contribute to this, the `confidence` value of the resulting statement is appropriately changed, furthermore the conflict situation is added to the `evidence` attribute. Further ideas like calculating the *average* value or the *maximum* value of the statement's `object` can be covered by the `function` attribute that allows to apply mathematical functions to the value of either one statement or a set of statements. The *function evaluation* forms the last part of the *control procedure*. Figure 6.16 shows several examples of conflict resolution strategies. In general, a strategy can be defined by every combination of resolvers, but not all make sense.

- 1.) `strategy=returnAll`
- 2.) `strategy=mostRecent(1)`
- 3.) `strategy=mostRecent(4)|mostConfident(3)|returnRandom(2)`
- 4.) `strategy=mostNamed(5)|mostConfident(3)|mostSpecific(1)`
- 5.) `strategy=mostPersonal(3)|mostConfident(2)&function=average`

Figure 6.16: Examples for conflict resolution strategies

The first conflict resolution strategy is the empty one, which means that no conflict resolution will be applied and all statements that pass the match and filter process will be returned. Note, if the `query.strategy` attribute is left empty, a default strategy will be applied but not necessarily the *returnAll* one. The second conflict resolution strategy is adequate for statements that are frequently renewed by only one sensor, thus simply the last entry is returned. The third strategy is slightly more complex: the four most recent statements are handed over and checked for the three most confident ones. However, only two statements of these three are returned by random selection. The fifth and last conflict resolution strategy first selects the most personal ones and then the most confident ones and finally applies the `average query.function` to calculate the average value of the two remaining statement's object. This strategy could be interesting for user model dimensions that do not change over time like *personality traits*.

If we revisit the prefacing example of figure 6.14 and apply the strategies of figure 6.16, the second and the third strategy would return *high*, the fourth strategy would return *medium* while the fifth strategy would return *low*. However, the first strategy returns all three values but is no help in this situation. These completely different results show the power of the conflict resolution strategies, however they also indicate that not all conflicting statements can be satisfactorily solved by resolution strategies. As different strategies lead to different statements and resulting values, the choice of the “right” conflict resolution strategy isn't that obvious.

¹The idea of *clarification by dialog* was recommended by Vania Dimitrova.

Different classes of problems seem to need different conflict resolution strategies. The open question is if there is any correlation of the best strategy in certain circumstances. We expect that already the user model dimension equivalence classes correlate with the best strategies. If so, this information could be added to the general user model ontology GUMO and the problem could be solved with the introduction of a new level of meta rules.

6.3.4 Conflict Ranking for Retrieved Statements

If several statements are returned, the `query.ranking` attribute controls the order of the statements. Three example rankings are presented in figure 6.17. They show that the same terms are used in the `query.ranking` and in the `query.strategy` attribute, however with a different impact.

- 1.) `ranking=mostRecent`
- 2.) `ranking=mostRecent|mostConfident`
- 3.) `ranking=mostNamed|mostSpecific|mostPersonal`

Figure 6.17: Example rankings for situation retrieval

The first ranking orders the resulting statements according to a temporal axis, the second ranking adds to the temporal axis the second level ordering on a confidence axis, and the third example ranking orders the statements under completely different settings. The so far discussed conflict resolution topics left the semantics of the statement's `mainpart` out of consideration. The following subsection integrates both: conflict resolution with semantic analysis.

6.3.5 Conflict Detection and Classification of Semantical Conflict Sets

In the prefacing chess example of subsection 6.3.2 the conflict was easy to detect since all three statements used the same `subject`, the same `auxiliary`, the same `predicate` and even the same `range=low-medium-high`. If one statement used instead the range *poor-average-good-excellent* a so-called SEMANTICRANGEMAPPING had to be executed, where for example *poor* is mapped to *low*, *average* and *good* are mapped to *medium*, and *excellent* is mapped to *high*. A more tricky problem is how can we detect conflicts if even the `auxiliary` and `predicate` in the conflicting statements (or in the query and the statements) differ even though the same ontology is used? For example, system B could use `auxiliary=hasKnowledge` and `predicate=chess` while system C uses `auxiliary=hasAbility` and `predicate=boardGames`. In such a case, we already need SEMANTICPROPERTYMAPPING². See [Ram and Park, 2004] for a discussion of a semantic conflict resolution ontology. A closely related problem arises if statements refer to different ontologies. In such a case we additionally need ONTOLOGYMAPPING in order to be able to detect and solve these conflicts.

²PROPERTY is used here to denote the pair of `auxiliary` and `predicate`

For the challenge of detecting possible conflicts we use a technique which is also used in production systems: we classify statements into so-called *conflict sets*. Mathematically speaking these statements are classified into equivalence classes. The following listing analyzes several types of such equivalence classes for conflict resolution with SITUATIONAL-STATEMENTS and arranges them into six levels with increasing computational load.

Conflict Detection Class 0: $\langle S, A, P, R \rangle$

Statements with the same subject (S), auxiliary (A), predicate (P) and range (R) are called to be in a potential conflict of level 0. These conflicts are easy to detect since only a syntactical comparison is necessary. However, every fast changing user model dimension like *heart beat* for example would constantly produce potential conflicts and thus the produced conflict sets are not very useful.

Conflict Detection Class 1: $\langle S, A, P, R \rangle_{\substack{\text{nonExpired} \\ \text{nonReplaced}}}$

The problem of the *conflict detection class 0* is solved by removing all statements that are already expired or replaced. The following simple conflicts that only differ in the object value can be detected with the *conflict detection class 1*:

- 1.1) subject=*Tim* auxiliary=*hasProperty* predicate=*happy* range=*yesNo* object=*yes*
- 1.2) subject=*Tim* auxiliary=*hasProperty* predicate=*happy* range=*yesNo* object=*no*

Conflict Detection Class 2: $\langle S, A, P, R^* \rangle_{\substack{\text{nonExpired} \\ \text{nonReplaced}}}$

The conflict sets that are defined by the *conflict detection class 2* only match against subject, auxiliary and predicate while the ranges R^* are transformed into each other with the SEMANTICRANGEMAPPING. The mappings between different ranges are defined in the GUMO ontology. The following two conflicting example statements differ in range and object.

- 2.1) subject=*Tim* auxiliary=*hasProperty* predicate=*happy* range=*yesNo* object=*yes*
- 2.2) subject=*Tim* auxiliary=*hasProperty* predicate=*happy* range=*lowHigh* object=*low*

Conflict Detection Class 3: $\langle S, A, P^*, R^* \rangle_{\substack{\text{nonExpired} \\ \text{nonReplaced}}}$

The conflict sets that are defined by the *conflict detection class 3* can differ in the predicate, range and object values. P^* indicates that related predicates are classified together. This relation could for example be defined as *synonyms* (other predicates with the same meaning) or *hypernyms*³ (concepts with a broader meaning) or *holonyms* (the whole concept of which the predicate is part of) or *opponyms* (concepts with opposite meanings like *happy* versus *sad* or *extraverted* versus *introverted*).

- 3.1) subject=*Tim* auxiliary=*hasProperty* predicate=*happy* range=*yesNo* object=*yes*
- 3.2) subject=*Tim* auxiliary=*hasProperty* predicate=*sad* range=*lowHigh* object=*high*

Conflict Detection Class 4: $\langle S, A^*, P^*, R^* \rangle_{\substack{\text{nonExpired} \\ \text{nonReplaced}}}$

The *conflict detection class 4* is only a slight extension to level 3, where additionally the auxiliary attributes are semantically related. For example *hasLearned* and *hasKnowledge* are closely related. Also, if someone *hasInterest* in something, one can assume that at least a rudimentary *hasKnowledge* statement also holds. Furthermore if

³Compare the WordNet relations that are defined in subsection 3.3.3 on page 45

a *hasPreference* relation holds, a *hasInterest* relation should also hold. However, this level 4 has not yet been used for concrete conflict resolution.

Conflict Detection Class 5: $\langle S^*, A^*, P^*, R^* \rangle_{nonReplaced}^{nonExpired}$

Interesting about level 5 is that the statements can completely differ syntactically but still be recognized as being semantically conflicting. The newly extended S^* integrates the modeling of user groups. If the subject defines a group or is member of a group, the corresponding GROUPMEMBERMAPPING has to be applied in order to handle conflicts between “member-statements” and “group-statements”. Furthermore S^* allows to handle the problem that one user is often identified in different systems with different logins or nicknames⁴.

The denotations $\langle S, A, P, R \rangle$ to $\langle S^*, A^*, P^*, R^* \rangle_{nonReplaced}^{nonExpired}$ are called the *signatures* of the corresponding equivalent classes.

To summarize, in this subsection we have demonstrated that the complex task of detecting conflicting statements in a repository can be analyzed with a fine-grained definition of conflict sets, that represent syntactical as well as semantical conflicts. Ontological reasoning is mostly integrated by using semantic mapping functionality. However, further research (like the experimental analysis of the conflict detection classes) has to be done at this point.

6.3.6 Architecture of Smart Situation Retrieval

The architectural diagram in figure 6.18 shows the SMARTSITUATIONRETRIEVAL process. The focus is set on the semantic conflict resolution part. It represents the control macro-step of the query process as shown in figure 6.2.

The oval numbers indicate the reading direction.

- (1) shows the request in UserQL that has to be parsed first.
- (2) points to the distributed retrieval of SITUATIONALSTATEMENTS.
- (3) summarizes the three macro-steps *select*, *match* and *filter* and presents the FILTERINGRESULT as input to the conflict resolution process.
- (4) stands for the three syntactical procedures VARIATIONMAPPING, REMOVEEXPIRED and REMOVEREPLACED.
- (5) shows the three semantical procedures GROUPMEMBERMAPPING, SEMANTICPROPERTYMAPPING and SEMANTICRANGEMAPPING that are based on knowledge in the ontologies GUMO, UbiWorldOntology, SUMO/MILO and the knowledge base WordNet.
- (6) shows the detection of conflicts and the construction of $\langle S^*, A^*, P^*, R^* \rangle_{nonReplaced}^{nonExpired}$ conflict sets.
- (7) points to the post-processing of *ranking*, *format*, *naming* and *function* that control the output format.
- (8) forms the resulting UserML report, that is sent via HTTP to the requestor.

⁴ S^* multi-name example: In one implemented ubiquitous user modeling scenario, the positioning system (see section 9.4.2) used the device’s IP for identification, UbiWorld used an internal UbiId, YAMAMOTO (see section 8.12) modeled it by URI while the SPECTER shopping assistant also used a different ontology representation.

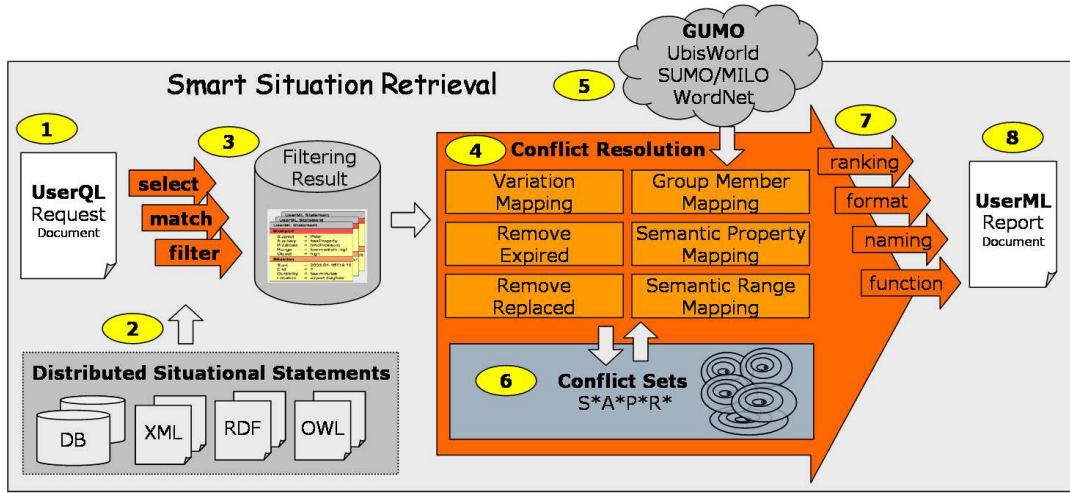


Figure 6.18: Smart situation retrieval with focus on semantic conflict resolution

6.4 Discussion and Design Decisions

In summarizing this chapter about information retrieval and conflict resolution, we will now discuss our design decisions: one design idea behind the user model query language *UserQL* was that it should be syntactically simple (realized as a flat structure) and close to the user model markup language *UserML*. The knowledge about the functionality should analogously be defined in an ontology such that the semantics of the queries' attributes and the statements' attributes are uniform. A tricky challenge was the integration of constraints of an XML query language, SQL query language and URI query language, since all three instances had to be realized by the general model of *SITUATIONALQUERIES*. However, the most important idea was to integrate syntactic and semantic conflict resolution into the query language.

Even though the so-called *SMARTSITUATIONRETRIEVAL* integrates various techniques of information retrieval and ontological reasoning, the dominating point of view should still be *information retrieval* and not the *ontology*. That is to say the query process which is mostly applied to databases with *SITUATIONALSTATEMENTS* dominates in our approach the ontological inference. Our situation models, that are part of the A-boxes, may be full of inconsistencies that need not to be resolved in general since we do not intend to change the statement repositories into one consistent world model. The statements that have been collected and sent around won't be changed or deleted. The reason is that in the highly distributed architecture, which is described in the next section, information is passed on and in most cases, we don't have access rights to edit the various instances of the statements. However, we allow to comment on them by reifying a statement into a new one and we allow a statement to be marked as being replaced. Furthermore we allow hiding them with the integrated privacy mechanism. The first advantage is that each user and each requesting system can individually choose the preferred conflict strategies and receive its individual model. The second advantage is demonstrated in the next chapter where we see that the integration of partial, decentralized user models can directly be realized using the *SMARTSITUATION-*

RETRIEVAL. The third advantage is that privacy handling is not the only mechanism that leads to individual models which results in better privacy.

The challenge that different users and different systems may want to use individualized ontologies that control the inferences could be solved in the following way: a new attribute with the name `ontology` is added to the *Control Box* (as defined on page 121) that carries one or several URIs pointing to the semantic web representations of these ontologies. This means that not only the set of statements differ individually and the conflict resolving strategies differ individually but also the inference ontologies can be chosen individually.

Another issue: how should we apply for example a confidence filter, if several statements in a conflict set don't carry confidence values? Or how should we apply the recency filter if the temporal constraints are not given? The answer is that the conflict resolution can only be as good as the available set of statements.

